Yazılım Testi: Katmanlı Regresyon Stratejisi

ÖFY

November 2026

Özet

Bir haberin kalite kontrolü sabit bir süre alır. Haber sayısı arttıkça, her değişiklikten sonra bütün haberleri test etmek uzar ve geliştirmeyi yavaşlatır. Bunun yerleşik çözümü testi iki katmana ayırmaktır. Değişiklikten hemen sonra küçük ve hızlı bir test koşar, geniş test ise arka planda ya da gece koşar. Google, Meta, Mozilla, Rust ve PostgreSQL bu yapıyı farklı biçimlerde kullanır. Ancak Google'da geniş test, kod ana depoya girdikten sonra ama müşteriye çıkmadan önce koşar. "Önce gönder, sonra test et" yaklaşımı ancak hızlı bir geri dönüş mekanizmasıyla birlikte güvenlidir. Tek bir haberle yapılan test, yalnızca o habere ait bir değişiklikte yeterlidir. Ortak kod değiştiğinde sabit ve çeşitli bir test seti (altın set) gerekir. Test önbelleği ve paralel çalıştırma, geniş testin süresini kısaltır. Savunma ve havacılık sektörü de aynı yöntemleri kullanır, ancak can güvenliği söz konusu olduğunda tam test bitmeden yazılım sahaya çıkmaz.

Giriş

Regresyon testi (*regression testing*), bir değişiklikten sonra daha önce çalışan bir şeyin bozulup bozulmadığını kontrol etmektir. **Kapalı döngü testi** ise sistemin gerçek bir iş üzerinde baştan sona çalıştırılması ve çıktısının beklenen sonuçla karşılaştırılmasıdır. Haber videosu üreten bir sistemde bu, gerçek haberlerin üretilip kalite kontrolünden geçirilmesi anlamına gelir.

Sorun basittir. Bir haberin kontrolü yaklaşık 40 saniye sürüyorsa, 100 haberlik bir test bir saatten uzun sürer. Her değişiklikten sonra bu süreyi beklemek, küçük bir düzeltmenin müşteriye ulaşmasını saatlerce geciktirir. Testi tamamen kaldırmak da çözüm değildir, çünkü bozulmaları müşteriden önce yakalayan şey bu testtir.

Yaygın bir yanlış kabul, "yalnızca değişen haberi test etmek" ile **etki analizi** (*test impact analysis*) yapmanın aynı şey olduğudur. Etki analizi, bir değişikliğin hangi testleri etkileyebileceğini bağımlılıklara bakarak belirler. Değişiklik ortak bir parçadaysa, örneğin kalite kontrolün kendisindeyse, etkilenen şey bütün haberlerdir. Bu durumda tek bir haberle yapılan test etki analizi değil, yalnızca bir duman testidir (*smoke test*).

Bu not test süresini kısaltma yöntemlerini, yayın stratejisini, gece testini ve bu yöntemlerin sektördeki kullanımını kapsar. Belirli bir test aracının kurulumu kapsam dışındadır.

Değişikliğe Göre Test Kapsamını Daraltma

Presubmit ve Postsubmit Ayrımı

Google'ın test altyapısı TAP (*Test Automation Platform*) testi iki aşamaya ayırır. Gönderimden önceki aşamada (*presubmit*) yalnızca değişiklikle doğrudan ilgili testler koşar. Gönderimden sonraki aşamada (*postsubmit*) daha geniş testler, birikmiş değişiklikler üzerinde toplu olarak koşar. Bir bozulma çıkarsa, otomatik araçlar bozulmaya hangi değişikliğin yol açtığını bulur (Google Research, 2023).

Bu ayrımın gerekçesi, her değişiklikte her şeyi test etmenin pratik olmamasıdır. Google, artan test yükünün hem bekleme süresini hem makine maliyetini büyüttüğünü belirtir ve bu yüzden hangi testlerin bozulma ihtimalinin yüksek olduğunu tahmin eden yöntemler geliştirmektedir (Google, 2025).

Tahmine Dayalı Test Seçimi

Meta (Facebook), geçmiş test sonuçlarından eğitilen bir model kullanır. Model, bir değişiklik ile bir testin özelliklerine bakarak o testin bozulma olasılığını tahmin eder. Bu yöntemle, bağımlılık grafiğinin seçeceği testlerin üçte birinden azı koşturulur. Buna rağmen tek tek test hatalarının %95'inden fazlası ve hatalı değişikliklerin %99,9'undan fazlası yakalanır. Test maliyeti de yarıya iner (Machalica vd., 2019).

Bu yöntem büyük ölçekli kod tabanları için tasarlanmıştır. Az sayıda haber içeren bir sistemde gereksizdir. Ancak haber sayısı çok büyüdüğünde bir sonraki adım olarak değerlendirilebilir.

Seçimin Riski

Değişikliğe göre test seçmenin bilinen riski, dolaylı bağımlılıkları kaçırmaktır. Bir parçadaki değişiklik, birkaç adım ötedeki başka bir parçayı bozabilir. NASA'nın yazılım mühendisliği rehberi de test seçimini etki analizine dayandırmayı önerir, ancak güvenlik kritik yazılımlarda bu analizin özellikle titiz olmasını ister. Arka planda koşan geniş testin işlevi, bu kaçırılan durumları yakalamaktır.

Değişikliğin Türüne Göre Karar

Pratik bir kural, değişiklikleri ikiye ayırmaktır:

- **Habere özgü değişiklik:** Yalnızca bir haberin metni, kesimi ya da alıntısı değişmiştir. Bu durumda yalnızca o haberi test etmek yeterlidir, çünkü değişiklik başka bir yere dokunmaz.
- **Ortak kod değişikliği:** Kalite kontrol, üretim hattı ya da şablon değişmiştir. Bu durumda değişiklik bütün haberleri etkiler ve tek haber yetmez. Sabit bir test seti kullanılmalıdır.

Örnek: Kalite kontrolde alıntıların dinlenmesi ile görüntünün incelenmesi sırayla çalışıyorsa ve bu iki adım paralel hâle getiriliyorsa, değişiklik ortak koddadır. Bu durumda sadece bir haberle test yapmak yanıltıcı bir güven verir.

Altın Set

Tanım ve Köken

Altın set (*golden reference*), sabit tutulan bir test girdisi kümesi ile bu girdilerde bilinen iyi bir sürümün ürettiği sonuçların birlikte saklanmasıdır. Yeni sürüm aynı girdilerle çalıştırılır ve sonuçlar kayıtlı olanlarla karşılaştırılır. Yöntem yazılımdaki *golden master* ya da onay testi (*approval testing*) fikrinden gelir. Bu fikir Michael Feathers tarafından adlandırılmıştır (Feathers, 2004).

Altın set bir çıktının "doğru" olup olmadığını değil, "değişmeden kalıp kalmadığını" ölçer. Bu yüzden kayıtlı sonuçların gerçekten iyi bir sürümden alınmış olması gerekir.

Video Sektöründeki Kullanım

Video kalitesi alanında bu yöntem standarttır. Kodlama hattında bir değişiklik yapıldığında, sabit test klipleri yeniden işlenir ve kalite skorları kayıtlı skorlarla karşılaştırılır. Skor belirli bir eşiğin altına düşerse değişiklik birleştirilmez. Netflix'in geliştirdiği VMAF (*Video Multimethod Assessment Fusion*) bu karşılaştırmada yaygın olarak kullanılan bir kalite ölçüsüdür (Forasoft, t.y.).

Kurulum İlkeleri

- **Çeşitlilik:** Tek bir klip bir test seti değildir. Set, sistemi zorlayan farklı türleri kapsamalıdır. Haber örneğinde bunlar çok alıntılı bir haber, görüntüsü ağır bir haber, kısa bir haber, uzun bir haber ve bilinen bir kenar durumu olabilir.
- **Sabitlik:** Set her seferinde aynı kalır. Böylece sonuçlar karşılaştırılabilir olur.
- **Tolerans:** Yapay zekâ ya da kalite ölçüsü kullanan testlerde sonuçlar her çalıştırmada biraz değişir. Bu yüzden tam eşitlik aranmaz, ölçümün kendi oynaklığına göre belirlenen bir tolerans payı bırakılır. Pay çok dar olursa test sürekli boşuna kırmızı yanar ve ekip ona güvenmeyi bırakır.
- **Boyut:** Üç ila beş girdi genellikle birkaç dakikada tamamlanır ve paralel çalıştırılabilir.

Test Önbelleği ve Paralellik

Önbellek

Google'ın derleme aracı Bazel, test sonuçlarını önbellekte tutar. Bir testin kodu ve bütün girdileri bir önceki çalıştırmadan beri değişmediyse, test yeniden koşturulmaz ve eski sonuç kullanılır. Bazel varsayılan olarak yalnızca başarılı sonuçları saklar. Başarısız testler her seferinde yeniden koşar (VirtusLab, t.y.).

Haber testine uyarlandığında önbellek anahtarı iki parçadan oluşmalıdır:

- Haberin girdilerinin özeti (*hash*)
- Kalite kontrolü ve üretim hattını oluşturan kodun sürümü

Bu yapının önemli bir sonucu vardır: ortak kod değiştiğinde anahtarın ikinci parçası değişir ve bütün haberler yeniden test edilir. Dolayısıyla önbellek, testi her gün kısaltmaz. Yalnızca içerik değişip ortak kodun değişmediği günlerde kısaltır.

Paralellik

Haberler birbirinden bağımsız test edilebiliyorsa aynı anda çalıştırılabilir. Sekiz haberi aynı anda çalıştırmak, süreyi kabaca sekizde birine indirir. Tek bir haberin içindeki bağımsız adımlar da paralel hâle getirilebilir. Örneğin alıntı kontrolü ile görüntü kontrolü birbirini beklemiyorsa, aynı anda çalıştırılarak bir haberin süresi yaklaşık 40 saniyeden 20 saniyeye indirilebilir.

Yayın Stratejisi

Gönderim ile Yayın Ayrımı

Google'ın postsubmit testi, kod ana depoya girdikten sonra koşar. Ancak ana depoya girmek müşteriye çıkmak demek değildir. Yayın ayrı bir aşamadır. Dolayısıyla "önce müşteriye gönder, sonra tam test et" düzeni Google'ın yöntemi değildir. Bu düzende tam test bir bozulma bulduğunda, müşteri bozuk sürümü zaten kullanıyor olur.

Hız gerektiğinde bu düzen kullanılabilir, ancak yalnızca hızlı bir geri dönüş mekanizmasıyla birlikte.

Kanarya Yayını

Kanarya yayını (*canary release*), yeni sürümün önce küçük bir kesime verilmesi, izlenmesi ve sorun yoksa yavaş yavaş herkese açılmasıdır. Adı, madencilerin zehirli gazı fark etmek için madene kanarya götürmesinden gelir.

Argo Rollouts ve Flagger gibi araçlar bu süreci otomatikleştirir. Flagger örneğinde trafiğin %10'u yeni sürüme verilir, ölçümlere bakılır, iyi giderse oran artırılır, kötü giderse eski sürüme dönülür (OneUptime, 2026). Bir rehbere göre hata oranı normalin iki ila üç katına çıktığında geri dönüş, insan beklenmeden otomatik başlamalıdır (Matalka, 2025).

Haber sisteminde karşılığı şudur: yeni kod önce birkaç gerçek haberi işler, sorun yoksa bütün haberlere geçer.

Geri Dönüş ve Özellik Bayrakları

Geri dönüş tek adımda yapılabilmelidir. Önceki sürüm hazırda tutulur ve sorun çıktığında önce ona dönülür, düzeltme sonra yapılır. Böylece müşterinin bozuk sürümü görme süresi saatlerden dakikalara iner.

Özellik bayrağı (*feature flag*) ise kodun yayınlanmasını özelliğin açılmasından ayırır. Kod gönderilir ama yeni davranış bir ayar açılana kadar devreye girmez (Matalka, 2025).

Gece Testi

Sektördeki Örnekler

Gün içinde hızlı test, gece ya da periyodik olarak geniş test yaygın bir düzendir:

- **Mozilla:** Her gün bir Firefox Nightly sürümü çıkar ve yeni sürümlere karşı otomatik testler koşar. Bir bozulma bulunduğunda `mozregression` aracı gecelik sürümler arasında arama yaparak bozulmanın hangi gün ve hangi değişiklikle geldiğini daraltır (Mozilla Wiki, t.y.).
- **Rust:** Derleyici değişikliklerinden önce Crater aracı çok sayıda gerçek Rust projesini derler, testlerini koşturur ve sonuçları eski ve yeni derleyici arasında karşılaştırır (Rust Project, t.y.). Bu, gerçek iş üzerinde eski ve yeni sürümü kıyaslayan bir kapalı döngü testidir.
- **PostgreSQL:** Build farm, gönüllülerin makinelerinden oluşan dağıtık bir sistemdir. Koddaki değişiklikler farklı platformlarda otomatik test edilir ve sonuçlar tek bir yerde toplanır (PostgreSQL Global Development Group, t.y.).

Bu örneklerin hiçbiri bir geliştiricinin dizüstü bilgisayarına dayanmaz. Testler sunucularda ya da ayrılmış makinelerde koşar ve sonuçlar bir panoya yazılır.

Dizüstü Bilgisayarda Gece Testi

macOS'ta `caffeinate -i` komutu, verilen komut bitene kadar sistemin kendiliğinden uykuya geçmesini engeller. Ancak kapak kapatıldığında Mac yine uyur. Bunun tek istisnası, harici ekran ve güç bağlıyken kapak kapalı çalışma modudur. Test çıktısı da bir dosyaya yazılmalıdır, aksi hâlde terminal kapandığında sonuç kaybolur:

```
caffeinate -i python3 run.py > ~/gece-testi.log 2>&1
```

Bu düzende dikkat edilmesi gereken noktalar:

- Bir yapay zekâ asistanı gece kendiliğinden uyanıp sonucu okumaz. Sonucun sabah okunabilmesi için kayıt dosyası gerekir.
- Önbelleğin kodda gerçekten var olup olmadığı doğrulanmalıdır.
- Dış servis maliyeti tahminle değil, bir gecelik gerçek çalıştırmayla ölçülmelidir.

Sunucuda Gece Testi

Test bir dış API kullanıyorsa dizüstü bilgisayara bağlı değildir. GitHub Actions'ta zamanlanmış bir iş ya da ucuz bir sunucu, testi her gece otomatik koşturabilir. Bilgisayarın açık kalması gerekmez, sonuç her sabah kayıtlı olur ve test kırmızı çıktığında e-posta ya da bildirim gönderilebilir.

Gece Testinin Sınırı

Gece testi, gün içinde yayınlanan sürümü korumaz. Bir bozulmayı gece bulduğunda müşteri o sürümü saatlerdir kullanıyor olabilir. Bu yüzden gece testi, gündüzkü altın set ve geri dönüş mekanizmasının yerini tutmaz, yalnızca onları tamamlar.

Savunma ve Havacılık Sektöründe Kullanım

ABD Savunma Bakanlığı Yazılım Fabrikaları

ABD Hava Kuvvetleri 2017'de Kessel Run adlı yazılım fabrikasını kurdu. Ardından farklı fabrikaların ekiplerini birleştiren Platform One oluşturuldu (DVIDS, 2021). Bu yapılar her değişikliğin otomatik olarak derlendiği, test edildiği ve güvenlik taramasından geçtiği sürekli entegrasyon ve teslim (*CI/CD*) hatları kurdu. Bir Hava Kuvvetleri sunumuna göre yazılımın kullanım izni (*Authority to Operate*) eskiden ortalama 8 ay sürüyordu ve büyük ölçüde elle yapılıyordu. Yeni düzende hata ve güvenlik düzeltmelerinin haftalar yerine dakikalar içinde çıkması hedeflendi (Chaillan, 2019). Kara Kuvvetleri ve Deniz Piyadeleri de daha sonra kendi yazılım fabrikalarını kurdu (Red Hat, t.y.).

DO-178C ve Seçici Test

DO-178C, havacılık yazılımları için kullanılan sertifikasyon standardıdır. Standart regresyon testini açıkça şart koşmaz, ancak sektörde yaygın bir mühendislik uygulamasıdır. Parasoft gibi firmalar bu sektöre değişikliğe göre test seçen araçlar satar. Yine de belirsizlik olduğunda bütün testleri koşturmak sektörde varsayılan yaklaşımdır (Parasoft, t.y.).

Donanımlı Kapalı Döngü Testi

Donanımlı döngü testinde (*hardware-in-the-loop*, HIL) gerçek bir cihaz, çevresini taklit eden bir simülasyona bağlanır. Her yeni yazılım sürümünde aynı senaryolar yeniden koşturulur ve sonuçlar karşılaştırılır (DMC, t.y.). Bu, kapalı döngü testinin donanımlı hâlidir. Critical Software'in bir vaka çalışmasında, DO-178C'nin en üst seviyesindeki bir sistemin elle yapılan testi ortalama bir aydan uzun sürüyordu. Test otomatikleştirildiğinde süre iki buçuk güne indi ve daha önce mümkün olmayan regresyon testi yapılabilir hâle geldi (Critical Software, t.y.).

Haber Sistemiyle Farkı

Savunmada kritik yazılım, tam test tamamlanmadan sahaya çıkmaz, çünkü hata can kaybına yol açabilir. Bir haber sisteminde hata, bir haberdeki kusurdur. Bu fark, haber sistemi için "hızlı set, yayın, hazır geri dönüş, sonra tam test" düzenini makul kılar. Aynı düzen savunmanın kritik sistemleri için yeterli olmaz.

Önerilen Akış

1. Değişikliğin türü belirlenir: habere özgü mü, ortak kodda mı.
2. Habere özgü değişiklikte yalnızca o haber test edilir.
3. Ortak kod değişikliğinde altın set paralel olarak koşturulur.
4. Test yeşilse değişiklik yayınlanır. Önceki sürüm geri dönüş için hazırda tutulur.
5. Yeni sürüm önce birkaç gerçek haberde çalışır, sorun yoksa bütün haberlere açılır.
6. Tam test arka planda ya da gece, tercihen bir sunucuda koşar. Değişmeyen haberler önbellekten okunur.
7. Tam test kırmızı çıkarsa önce eski sürüme dönülür, sonra düzeltme yapılır.

Sonuç

Test süresi sorunu testin kaldırılmasıyla değil, katmanlara ayrılmasıyla çözülür. Hangi katmanın ne zaman yeterli olduğunu belirleyen şey, değişikliğin etki alanıdır. Habere özgü bir değişiklikte tek haber testi gerçekten etki analizidir. Ortak kod değişikliğinde ise aynı test yanıltıcı bir güvence verir.

İkinci çıkarım, test ile yayının ayrı aşamalar olduğudur. Büyük şirketlerin "sonra test et" dediği yer müşteriden öncedir. Bu sıralamayı değiştirmek, ancak geri dönüşün hızlı ve kolay olmasıyla telafi edilebilir. Bu yüzden geri dönüş mekanizması, test stratejisinin ayrılmaz bir parçasıdır.

Önbellek ve paralellik süreyi kısaltır, ama sınırları vardır. Ortak kod değiştiğinde önbellek boşalır ve test yine uzun sürer. Paralelliğin sınırı ise dış servislerin istek kotası ve maliyettir.

Açık kalan sorular şunlardır: altın setin hangi haberlerden oluşacağı, tolerans payının nasıl ölçüleceği ve haber sayısı hangi seviyeye ulaştığında tahmine dayalı test seçiminin anlamlı hâle geleceği. Türk savunma firmalarının bu konudaki uygulamaları bu notta incelenmemiştir.

Kaynaklar

- Chaillan, N. (2019). *DevSecOps: The DoD Software Factory* [Sunum]. U.S. Air Force. https://www.slideshare.net/slideshow/devsecops-the-dod-software-factory/194833747
- Critical Software. (t.y.). *Automating Closed-Loop Testing* [Vaka çalışması]. https://criticalsoftware.com/resource/automating-closed-loop-testing
- DMC. (t.y.). How Hardware-in-the-Loop Testing Reduces Risk in Aerospace and Defense Programs. https://www.dmcinfo.com/blog/51793/hardware-in-the-loop-testing-aerospace-risk-reduction/
- DVIDS. (2021). DoD software giants sign joint collaboration statement. https://www.dvidshub.net/news/printable/409382
- Feathers, M. (2004). *Working Effectively with Legacy Code*. Prentice Hall.
- Forasoft. (t.y.). Regression testing and golden references. https://www.forasoft.com/learn/video-quality/articles-vqm/regression-testing-golden-references
- Google. (2025). Speculative Testing at Google with Transition Prediction. *ICST 2025 Industry Track*.
- Google Research. (2023). What Breaks Google? *ICST 2023 CCIW*. https://research.google/pubs/what-breaks-google/
- Machalica, M., Samylkin, A., Porth, M. ve Chandra, S. (2019). Predictive Test Selection. *ICSE-SEIP*. arXiv:1810.05286.

- Matalka, M. (2025). Progressive Delivery Best Practices with Continuous Deployment. *Terrateam*. https://terrateam.io/blog/progressive-delivery-best-practices
- Mozilla Wiki. (t.y.). QA/Desktop Firefox/Walkthroughs/Automation. https://m.wiki.mozilla.org/QA/Desktop_Firefox/Walkthroughs/Automation
- OneUptime. (2026). How to Implement Canary Releases with Flux and Flagger. https://oneuptime.com/blog/post/2026-02-09-canary-releases-flux-flagger-gitops/view
- Parasoft. (t.y.). DO-178C Regression Testing. https://www.parasoft.com/learning-center/do-178c/regression-testing/
- PostgreSQL Global Development Group. (t.y.). PostgreSQL Build Farm. https://buildfarm.postgresql.org/
- Red Hat. (t.y.). Modernizing defense software factories. https://www.redhat.com/it/topics/cloud-computing/modernize-defense-software-factories
- Rust Project. (t.y.). Crater. https://github.com/rust-lang/crater
- VirtusLab. (t.y.). Test Caching. *Bazel Book*. https://bazel.virtuslab.com/book/1~2~1/